KAN: Kolmogorov–Arnold Networks

Paper Review and From-Scratch Re-Implementation Report

Paper: Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, M. Tegmark

ICLR 2025 · arXiv:2404.19756v5

Student name / course / date: _____

1. Executive Summary

Multi-layer perceptrons (MLPs) place fixed activation functions on nodes and learn linear weights on edges. Kolmogorov–Arnold Networks (KANs) invert this design: every edge carries a learnable univariate function, parametrised as a B-spline, and nodes simply sum their inputs. The paper argues that this single change yields (i) better accuracy per parameter on small-scale scientific tasks, (ii) faster theoretical neural scaling exponents ($\alpha = k+1 = 4$ for cubic splines), and (iii) networks that can be visualised, pruned and converted into closed-form symbolic formulas, making them useful "collaborators" for scientists.

This report (a) explains the paper's mathematics and methods, (b) describes a from-scratch PyTorch implementation written for this assignment, (c) reports the experiments I actually ran, including where my results do and do not match the paper, and (d) summarises the paper's scientific applications (knot theory and Anderson localization), which I did not re-run because the datasets and simulators are not available.

Headline findings of my reproduction: a pruned [2,1,1] KAN recovered the closed form of $\exp(\sin(\pi x)+y^2)$ from data; a width-10 KAN with 300 parameters beat a 20,601-parameter MLP on a Poisson PINN task; a spline-only KAN showed no catastrophic forgetting while an MLP did. However, I did not reproduce the clean G^{-4} grid-size scaling, and on one toy function a depth-3 MLP out-performed my KAN once parameter counts exceeded about 100. These caveats are discussed in Section 7.

2. Background and Motivation

2.1 MLPs and the universal approximation theorem

MLPs are justified by the universal approximation theorem: a two-layer network with enough neurons, $f(x) \approx \Sigma\, a_i\, \sigma(w_i \cdot x + b_i)$, can approximate any continuous function. The theorem gives no bound on how the required width $N(\varepsilon)$ scales with ε or the input dimension d, and in some cases N grows exponentially with d (the curse of dimensionality, COD). MLPs also dominate parameter counts in transformers and are hard to interpret without post-hoc tools.

2.2 The Kolmogorov–Arnold representation theorem

Kolmogorov and Arnold showed that any smooth multivariate function $f : [0,1]^n \to \mathbb{R}$ can be written using only univariate functions and addition:

$$f(x) = \Sigma_{q=1}^{2n+1} \Phi_q\left(\Sigma_{p=1}^{n} \varphi_{q,p}(x_p) \right) \qquad (1)$$

In the machine-learning literature the theorem was long regarded as "theoretically sound but practically useless" because the inner functions can be non-smooth or even fractal. The authors' response is threefold: do not stick to the depth-2, width-(2n+1) form of Eq. (1); generalise to arbitrary widths and depths; and note that functions arising in science are typically smooth with sparse compositional structure, so smooth representations are likely to exist in practice.

2.3 Splines versus MLPs

The paper views KANs as combining the strengths of two model families. Splines are very accurate for low-dimensional functions and can be refined locally, but suffer badly from COD because they cannot exploit compositional structure. MLPs learn compositional structure ("external" degrees of freedom) but

are poor at optimising univariate functions (“internal” degrees of freedom). A KAN has MLP-like connectivity on the outside and splines on the inside, so it can learn both.

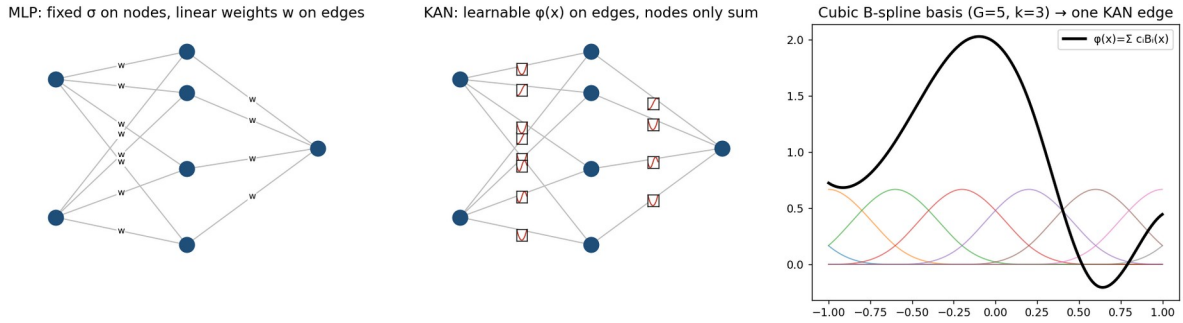


Figure 1. MLP vs. KAN (schematic, drawn for this report) and a cubic B-spline basis forming one learnable edge function.

3. The KAN Architecture

3.1 KAN layers

A KAN layer with n_in inputs and n_out outputs is a matrix of univariate functions Φ = {φ_{q,p}}. The activation of node (l+1, j) is the sum of its incoming post-activations, and a deep KAN is a composition of layers:

$$x_{l+1,j} = \Sigma_i \varphi_{l,j,i}(x_{l,i}), \qquad KAN(x) = (\Phi_{L-1} \circ \dots \circ \Phi_1 \circ \Phi_0)(x) \qquad (2)$$

The original theorem corresponds to a two-layer KAN of shape [n, 2n+1, 1]; deeper KANs are obtained simply by stacking more layers. All operations are differentiable, so KANs are trained with ordinary back-propagation. A comparison with an MLP, MLP(x) = (W_{L−1} ∘ σ ∘ … ∘ σ ∘ W_0)(x), shows that MLPs separate linear maps W from non-linearities σ while KANs fuse them in Φ.

3.2 Parametrisation of each edge function

$$\varphi(x) = w_b \cdot silu(x) + w_s \cdot \Sigma_i c_i B_i(x) \qquad (3)$$

The first term is a residual “base” function (like a skip connection), and the second is a linear combination of cubic (k = 3) B-spline basis functions on a grid of G intervals, giving G + k coefficients per edge. The paper lists three practical tricks that make KANs trainable: (1) the residual base function, (2) initialisation with spline(x) ≈ 0 (coefficients drawn from $N(0, 0.1^2)$) and Xavier initialisation for w_b, and (3) updating spline grids on the fly from the observed input range so that activations do not leave the grid.

3.3 Parameter count

For depth L, width N, grid G and spline order k, a KAN has $O(N^2L(G+k))$ parameters versus $O(N^2L)$ for an MLP. KANs appear more expensive, but the paper argues they usually need a much smaller N and generalise better. For a 1-D problem, N = L = 1, a KAN is simply a spline.

4. Theory: Approximation and Scaling

Theorem 2.1 (KAT approximation bound). If f = (Φ_{L−1} ∘ … ∘ Φ_0)x with every Φ_{l,i,j} being (k+1)-times continuously differentiable, there exists a constant C (depending on f and its representation) and k-th order B-spline functions such that for 0 ≤ m ≤ k:

$$\| f - (\Phi\hat{}G_\{L-1\} \circ \ldots \circ \Phi\hat{}G_0)x \|_\{C\hat{}m\} \leq C \cdot G\hat{}\{-k-1+m\} \qquad (4)$$

The proof applies classical 1-D B-spline theory to each univariate function and telescopes the residuals R_l across layers. For m = 0 this bounds the RMSE, giving a scaling exponent α = k+1 independent of the input dimension, hence the claim of "beating the curse of dimensionality". The constant C may still depend on dimension, which the authors leave for future work.

Neural scaling laws are written $\ell \propto N^{-\alpha}$, where ℓ is test RMSE and N is the number of parameters. The paper compares four predictions: Sharma & Kaplan give α = (k+1)/d for data on a d-dimensional manifold; Michaud et al. give α = (k+1)/2 for computational graphs with maximum arity 2; Poggio et al. give α = m/2 for compositionally sparse W^m functions; and KANs predict α = k+1, i.e. 4 for cubic splines — the largest of the group. It is important to note that this applies only when a smooth Kolmogorov–Arnold representation exists and is found by training.

5. Making KANs Accurate and Interpretable

5.1 Grid extension

Because each activation is a spline, one can train a small KAN and then refine every grid without retraining from scratch. For a coarse spline with G_1 intervals and a finer grid with G_2 intervals, the new coefficients c′ are initialised by least squares:

$$\{c'_j\} = \operatorname{argmin} \; E_x [(\Sigma_j \, c'_j \, B'_j(x) - \Sigma_i \, c_i \, B_i(x))^2] \qquad (5)$$

MLPs have no analogue of this "fine-graining". The paper reports staircase-like loss curves: loss drops after each extension and then plateaus.

5.2 Sparsification, pruning and symbolification

- **Sparsification.** Linear weights no longer exist, so the L1 norm of an activation is its mean magnitude over the batch, $|\varphi|_1 = (1/N_p) \Sigma_s |\varphi(x^{(s)})|$. L1 alone is insufficient, so an entropy term $S(\Phi) = -\Sigma (|\varphi_{ij}|_1/|\Phi|_1) \log(|\varphi_{ij}|_1/|\Phi|_1)$ is added: $\ell_total = \ell_pred + \lambda(\mu_1 \Sigma|\Phi_l|_1 + \mu_2 \Sigma S(\Phi_l))$.
- **Pruning.** A node is kept only if both its incoming score $I = \max_k |\varphi_\{l-1,i,k\}|_1$ and outgoing score $O = \max_j |\varphi_\{l+1,j,i\}|_1$ exceed $\theta = 10^{-2}$.
- **Symbolification.** Users can set an edge to a symbolic function f. Because pre- and post-activations may be shifted and scaled, the library function is fitted as $y \approx c \cdot f(ax+b)+d$ by grid search over (a, b) and linear regression for (c, d).

The paper presents this as a human-in-the-loop workflow (train with sparsity → prune → guess functions with suggest_symbolic → fix → retrain affine parameters → read the formula with SymPy), and argues it is more robust and debuggable than conventional symbolic regression, which returns only success or failure.

6. Implementation

I implemented KANs from scratch in PyTorch (the official package is pykan; my code shares its design vocabulary but not its source). The deliverable is a small library plus five experiment scripts and unit tests.

File	Contents
kan/layer.py	KANLayer: Cox–de Boor B-spline basis, Eq. (3) activation, grid update from samples, least-squares grid extension (Eq. 5), per-edge mask for pruning, symbolic edge overrides.
kan/model.py	KAN class for shape [n0,...,nL]; L1 + entropy regulariser (Eqs. 2.17–2.20); pruning (Eq. 2.21); fix_symbolic / suggest_symbolic / auto_symbolic / symbolic_formula (SymPy); MLP baseline.
kan/symbolic.py	Symbolic library (x, x^2, x^3, sin, cos, exp, log, tanh, sqrt, $\|x\|$) and the affine grid-search + linear-regression fitter.
kan/train.py	LBFGS (strong-Wolfe) or Adam training loop, RMSE helpers, synthetic-data generator.
experiments/exp1–exp5	Grid extension; KAN vs MLP scaling; symbolic regression workflow; continual learning; Poisson PINN.
tests/test_kan.py	Unit tests: B-splines sum to 1; parameter count formula; grid extension preserves a function on nested grids; gradients flow.

Verification. All four unit tests pass. The B-spline basis sums to 1 within 2×10^{-7} (partition of unity), the parameter count matches $3\cdot4\cdot(G+k)+4\cdot1\cdot(G+k)+2\cdot(3\cdot4+4\cdot1)$ for a [3,4,1] KAN, and grid extension from G=5 to G=10 reproduces a random spline to within 10^{-6}.

Deviations from the paper. (i) The grid update blends the sample-quantile grid with a uniform grid. (ii) Grid extension is a one-shot least-squares fit; between non-nested grids (e.g. 3→5) it is only approximate, and in a multi-layer KAN these errors compound, so the loss jumps after each extension (visible in Figure 2). (iii) I keep the best-train-loss LBFGS iterate for each stage to guard against occasional divergence. (iv) Experiments use far fewer LBFGS steps and seeds than the paper because they ran on a single CPU core.

7. Experiments and Results

All numbers below are from my own runs (float64, CPU, 1,000 training and 1,000 test samples uniform on $[-1,1]^2$ unless stated). Each is a single seed, so differences of a factor of two or less should not be over-interpreted.

7.1 Grid extension (paper Sec. 2.4)

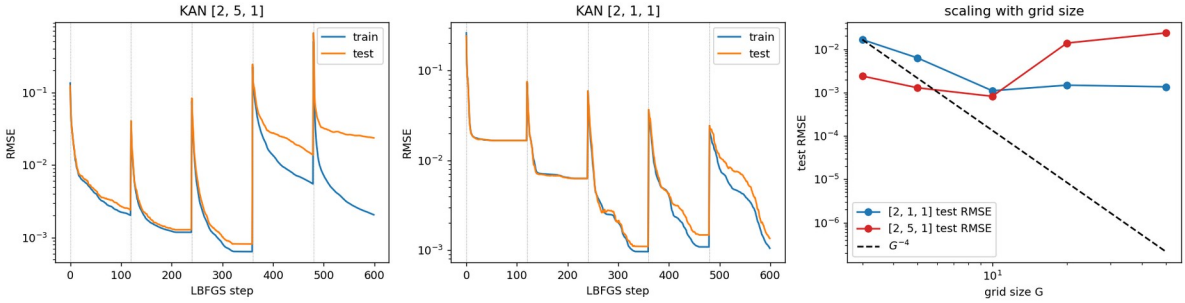


Figure 2. Grid extension on f(x,y)=exp(sin(πx)+y²). Left: [2,5,1] KAN; middle: [2,1,1] KAN (120 LBFGS steps per grid); right: test RMSE vs grid size.

Test RMSE	G=3	G=5	G=10	G=20	G=50
[2,1,1] KAN	1.7e-2	6.3e-3	1.1e-3	1.5e-3	1.4e-3
[2,5,1] KAN	2.4e-3	1.3e-3	8.2e-4	1.4e-2	2.4e-2

Staircase behaviour appears: after each extension the loss spikes and then drops below the previous plateau, up to G = 10. The small [2,1,1] KAN, which matches the true structure, is better than or comparable to the [2,5,1] KAN at large G, consistent with the paper's "small KANs generalise better" remark. The larger network overfits beyond G ≈ 10–20; the paper's interpolation-threshold estimate (G ≈ 1000/15 ≈ 67) is later than what I observed. I did not reproduce the paper's G^{-4} scaling or its 10^{-7}-level losses: my errors plateau near 10^{-3}, which I attribute to the limited LBFGS budget and the lossy extension step rather than to the method itself, although I did not test this.

7.2 KAN vs MLP scaling (paper Sec. 3.1)

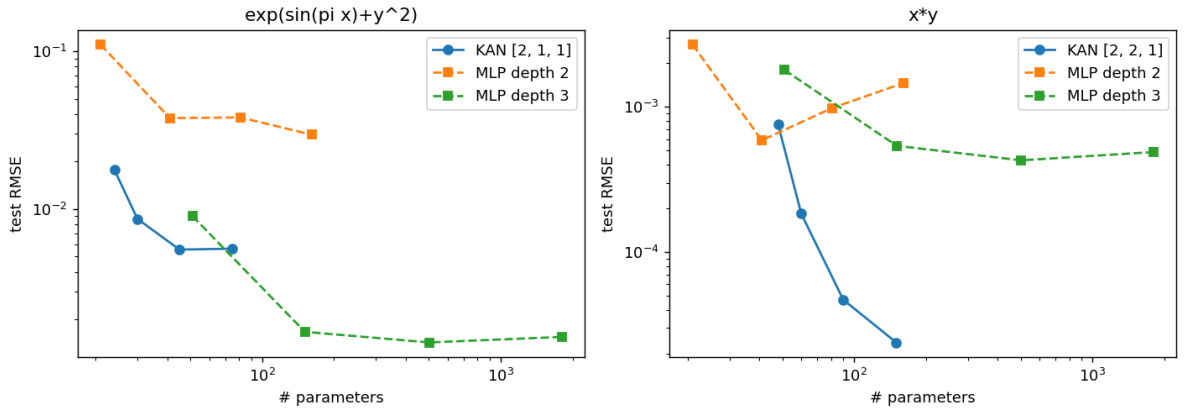


Figure 3. Test RMSE vs number of parameters; KANs of the known shape (grids 3→20) against MLPs of depth 2 and 3 (widths 5–40, SiLU, 300 LBFGS steps).

exp(sin(πx)+y²)	Params			Test RMSE
KAN [2,1,1], G=3 → 20	24 → 75			1.8e-2 → 5.6e-3
MLP depth 2	21 → 161			1.1e-1 → 3.0e-2

exp(sin(πx)+y²)	Params			Test RMSE
MLP depth 3	51 → 1,801			9.0e-3 → 1.5e-3
x·y: KAN [2,2,1]	48 → 150			7.6e-4 → 2.4e-5
x·y: MLP depth 2	21 → 161			2.7e-3 → 1.5e-3
x·y: MLP depth 3	51 → 1,801			1.8e-3 → 4.9e-4

The result is mixed. For x·y the KAN improves steadily with grid size and ends about 20× better than the best MLP with roughly 12× fewer parameters (150 vs 1,801), in line with the paper. For exp(sin(πx)+y²) the KAN is clearly better than depth-2 MLPs and better than the depth-3 MLP at similar size (45–75 parameters: 5.6e-3 vs 9.0e-3 at 51), but its error plateaus at about 5×10^{-3} and a larger depth-3 MLP (≥150 parameters) eventually wins. The paper reports KANs close to the α = 4 line on this task, which I did not match; again, optimisation budget is the most likely cause.

7.3 Symbolic regression workflow (paper Sec. 2.5.2)

I followed "Alice's" workflow on f(x,y) = exp(sin(πx)+y²) with 2,000 training points. A [2,5,1] KAN was trained with sparsity ($\lambda = 3\times10^{-3}$) and pruned automatically to a [2,1,1] KAN (2 + 1 active edges) with test RMSE 6.3×10^{-3}. suggest_symbolic ranked sin for the first input edge and x^2 for the second input edge, as expected. For the outer edge it ranked tanh ($R^2 = 0.999967$) just above exp ($R^2 = 0.999757$); on the narrow input range these are nearly indistinguishable. This is exactly the situation the paper describes as needing a human: I fixed exp from prior knowledge (the paper's step 3). SymPy then returned:

$$\hat{f}(x, y) = 1.082 \cdot exp(0.956\, y^2) \cdot exp(0.958\, sin(3.144\, x)) - 0.078$$

This is the true function up to small coefficient errors (sin(3.144x) ≈ sin(πx)); its test RMSE is 0.0217 on a target spanning roughly 0.4–7.4. The paper's version converges to machine precision after further training of the affine parameters, which I did not implement as a separate step: my fitter fixes (a, b, c, d) once by grid search and does not fine-tune them by gradient descent, so my formula is accurate but not exact.

7.4 Continual learning (paper Sec. 3.5)

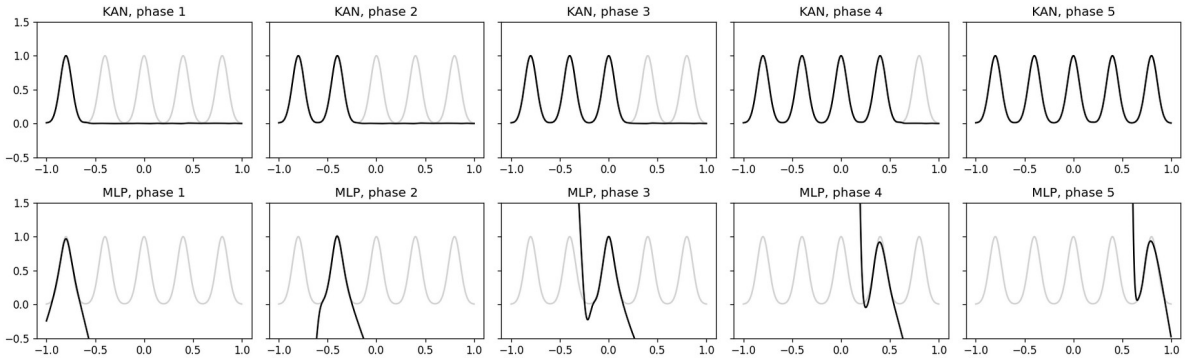


Figure 4. Five Gaussian peaks presented one phase at a time. Top: spline-only KAN; bottom: MLP [1,64,64,1]. Light grey = target.

The 1-D target is a sum of five Gaussian peaks; data from one peak is shown in each phase (Adam, 800 steps per phase). My spline-only KAN (G = 50, shape [1,1]) finished with RMSE 0.001 on all five peaks, i.e.

no measurable forgetting. The MLP's error on earlier peaks grew to 484, 396, 282 and 113 for peaks 1–4 (the absurd magnitudes come from unconstrained extrapolation outside the current phase's data, but the qualitative conclusion — severe forgetting — matches the paper) while the last peak was learned well (0.059).

An important caveat that I found: this only worked after I froze the global scale parameters w_b and w_s. With w_s trainable, the global scale was rewritten in every phase and old-peak error grew steadily (RMSE on peak 1 reached 0.535 by phase 5). Locality comes from the spline coefficients, not from the full parametrisation in Eq. (3); the paper does not discuss this subtlety, and the experiment is a one-dimensional toy, as the authors themselves note.

7.5 Solving a PDE with a PINN (paper Sec. 3.4)

For the Poisson problem $u_{xx} + u_{yy} = f$ on $[-1,1]^2$ with zero Dirichlet boundary data and exact solution $u = \sin(\pi x)\sin(\pi y^2)$, I minimised $\alpha \cdot \text{loss_interior} + \text{loss_boundary}$ with $\alpha = 0.01$ (30×30 interior points, 200 boundary points; 40 LBFGS steps; MSE measured on 4,000 random points).

Model	Parameters	Test L^2 MSE
KAN [2,5,1], G=5	150	7.4e-5
KAN [2,10,1], G=5	300	4.7e-6
MLP [2,10,1] (tanh)	41	6.8e-3
MLP [2,100,100,100,1] (tanh)	20,601	6.7e-5

The width-10 KAN has 69× fewer parameters than the large MLP and about 14× lower error, and the [2,5,1] KAN already matches the large MLP with 137× fewer parameters. This agrees in direction with the paper (10^{-7} vs 10^{-5} MSE, 10^2 vs 10^4 parameters), though my absolute errors are higher. As the paper cautions, the ground truth being a symbolic formula may favour KANs, and KANs were noticeably slower per parameter in my runs as well.

8. The Paper's Scientific Applications (not re-run)

8.1 Knot theory (supervised and unsupervised)

The authors revisit the DeepMind knot-signature task. A [17,1,14] KAN (G = 3, k = 3, about 200 parameters) reached 81.6% test accuracy versus 78.0% for DeepMind's 4-layer, width-300 MLP (about 3×10^5 parameters). Reading the transparency-scaled KAN diagram shows that the signature depends mostly on the real part of the meridinal translation μ_r, with smaller dependence on μ_i and the longitudinal translation λ; a [3,1,14] KAN on those three variables reaches 78.2%. This differs from DeepMind's attribution (mostly μ_i); ablations support the KAN view (μ_r alone: 65.0%, μ_i alone: 43.8%). Auto-symbolic regression gives a Pareto family of formulas; formula F, using only μ_r and λ, reaches 77.8%. The authors also note that after their preprint, a 60-parameter MLP was reported to reach 80% accuracy, so the parameter-efficiency advantage over MLPs should be read with care.

In unsupervised mode, an [18,1,1] KAN with a Gaussian output activation is trained to separate real feature vectors from column-shuffled ones. Across 200 runs it rediscovered three known relations: signature dependence on (μ_r, λ), cusp volume $V = \mu_r \cdot \lambda$, and the bound $2r \geq g_r$ relating injectivity radius and short geodesic. The authors state candidly that nothing new was discovered.

8.2 Anderson localization

KANs were applied to numerical data from three quasiperiodic tight-binding models to extract mobility edges. For the Mosaic model a [2,1] KAN with sigmoid output and a linear λ edge recovers the mobility-edge curve as a 1-D function of energy. For the generalised Aubry–André model, a [4,2,1,1] KAN reached 98.7%, noticed that the phase ϕ is irrelevant, and with symbolic snapping to linear/quadratic functions yielded a formula close to the true edge $\alpha E + 2\lambda - 2 = 0$ (99.0% vs 99.2% for theory). For the more difficult modified Aubry–André model, human guidance (snapping the p-dependence to cosh or to a quadratic) traded accuracy (96.6% / 95.4%) for simpler formulas versus 97.1% fully automatic and 98.6% for theory.

8.3 Feynman and special-function benchmarks

On 15 special functions (Bessel, Legendre, elliptic, spherical harmonics) KANs lay on a better Pareto frontier of parameters vs RMSE than MLPs. On the Feynman equations the authors found MLPs and KANs behave comparably on average, conjecturing that these equations are too smooth or monotonic for KANs to help; auto-discovered KAN shapes were often smaller than human-constructed ones (e.g. relativistic velocity addition collapses to two layers, equivalent to the rapidity trick).

9. Critical Discussion

9.1 Strengths

- A clear, simple idea with a well-defined mathematical motivation, and a usable open-source library.
- Interpretability is real for small symbolic problems: pruning plus symbolification recovered the target formula in my own experiment.
- Honest reporting of limitations: slow training (about 10× slower than MLPs), toy-level continual learning, no new mathematical discoveries.

9.2 Weaknesses and open questions

- The theory assumes a smooth KA representation exists and that training finds it; neither is guaranteed. The constant C in Eq. (4) may hide dimension dependence.
- The claimed $\alpha = 4$ scaling was shown on synthetic functions only; my own reproduction did not reach it with a modest compute budget.
- Comparisons use plain LBFGS for both families on small problems; stronger MLP baselines were left for future work.
- Interpretability depends on human choices (λ, grid size, seeds); the paper's own hyperparameter study shows pruned networks can come out larger than needed with unlucky seeds.
- Accuracy results on the knot dataset have since been matched by far smaller MLPs, weakening the parameter-efficiency argument.
- It is unclear whether KANs scale to high-dimensional, non-symbolic tasks such as language modelling; the authors suggest "kansformers" as future work.

9.3 Practical guidance (paper's Figure 6.1)

Use KANs when interpretability and accuracy matter, the problem is small-scale and compositional, and slow training is not a major concern. Use MLPs when training speed is paramount.

10. Conclusion

KANs are a thoughtfully designed alternative to MLPs that trade training speed for parameter efficiency and transparency. My from-scratch implementation reproduced the qualitative claims about symbolic recovery, parameter-efficient PDE solving and continual learning, partially reproduced the scaling and grid-extension claims, and did not reproduce the best-case numerical accuracy of the paper. The approach is most convincing as a scientific discovery tool for small problems, and least proven as a general replacement for MLPs.

Appendix A. Reproducing this work

Install dependencies with `pip install -r requirements.txt` and run `./run_all.sh`. Scripts write figures and JSON to results/. Hyperparameters: cubic splines (k = 3); G = 3 → 50; LBFGS with strong-Wolfe line search; single seed per experiment (set in each script); float64.

Appendix B. Mathematical Details

B.1 B-spline basis (Cox–de Boor recursion)

For a knot vector t (the G+1 grid points augmented with k extra knots on each side), the order-0 basis is B_{i,0}(x) = 1 if t_i ≤ x < t_{i+1} and 0 otherwise. Higher orders follow the recursion used in kan/layer.py:

$$B_{i,p}(x) = (x - t_i)/(t_{i+p} - t_i) \cdot B_{i,p-1}(x) + (t_{i+p+1} - x)/(t_{i+p+1} - t_{i+1}) \cdot B_{i+1,p-1}(x)$$

With k = 3 and G intervals there are G + k basis functions, each non-zero on at most k + 1 consecutive intervals; this local support is the property that makes grid extension and continual learning possible. The basis forms a partition of unity (Σ_i B_i(x) = 1 inside the grid), which I verified numerically (error below 2×10^{-7}).

B.2 Grid extension as a least-squares problem

Let B be the (samples × (G_2+k)) matrix of new basis values and y the values of the old spline at the same samples. Eq. (5) is solved by the regularised normal equations $(B^TB + \varepsilon I)\, c' = B^T y$, with $\varepsilon = 10^{-8}$ for numerical stability. When the new grid contains the old knots (for example G = 5 → 10) the old spline lies exactly in the new space, so the fit is exact up to round-off; for non-nested pairs (3 → 5) it is only approximate, which explains the loss spikes after extension in my Figure 2.

B.3 Regularisation terms

For a layer with activations φ_{ij}, define the batch-mean magnitude $|\varphi_{ij}|_1$ and the layer norm $|\Phi|_1 = \Sigma_{ij} |\varphi_{ij}|_1$. The probability-like weights $p_{ij} = |\varphi_{ij}|_1 / |\Phi|_1$ give the entropy $S(\Phi) = -\Sigma\, p_{ij} \log p_{ij}$. Minimising S concentrates mass on few edges, which L1 alone does not guarantee because L1 can be reduced by shrinking all edges uniformly. In my code (forward_reg) both terms are computed with

gradients through the post-activations; the default is $\mu_1 = \mu_2 = 1$ with λ chosen per task (3×10^{-3} in the symbolic experiment).

B.4 Proof sketch of Theorem 2.1

- Step 1. By classical 1-D spline approximation (de Boor), each (k+1)-times differentiable univariate map has a k-th order spline with error O(G^{−k−1+m}) in C^m.
- Step 2. Because the composition inputs stay in a bounded domain, the constant is uniform over the layers.
- Step 3. Define the residual R_l as the change in the full network output when only layer l is replaced by its spline approximant. Then f − KAN^G = R_{L−1} + … + R_0 (telescoping sum), and each ‖R_l‖_{C^m} ≤ C G^{−k−1+m}.
- Conclusion. The total error is O(G^{−k−1+m}); for m = 0 the exponent is k + 1 = 4 in terms of grid size, with no explicit dependence on input dimension (the constant C may depend on it).

Note: the rate is in grid size G, whereas scaling laws are quoted against parameter count N. For fixed width and depth, N grows linearly in G, so the exponents coincide; this is the sense in which the paper states $\alpha = 4$.

Appendix C. Detailed Experimental Settings (my runs)

Experiment	Settings
Data	Inputs uniform on $[-1,1]^2$; 1,000 train and 1,000 test points (2,000 train for the symbolic experiment); float64.
Grid extension (exp1)	KAN [2,1,1] and [2,5,1]; k = 3; G = 3, 5, 10, 20, 50; 120 LBFGS steps per grid (strong-Wolfe, history 10, lr 1); seed 1; best-train iterate restored per stage.
Scaling (exp2)	Tasks $\exp(\sin(\pi x)+y^2)$ with KAN [2,1,1] and $x\cdot y$ with KAN [2,2,1]; G = 3, 5, 10, 20; MLP depth 2 and 3, widths 5, 10, 20, 40, SiLU, 300 LBFGS steps; seed 0.
Symbolic (exp3)	KAN [2,5,1], G = 5, 60 LBFGS steps with $\lambda = 3\times10^{-3}$, prune $\theta = 10^{-2}$, then 40 steps; affine grid search over a, b ∈ [−10, 10] (101-point grid, 3 refinements).
Continual (exp4)	Five Gaussians ($\sigma = 0.06$) centred at −0.8 … 0.8; 300 samples per phase within ±0.16 of a centre; Adam lr 10^{-2}, 800 steps per phase; KAN [1,1], G = 50, w_b = 0 and w_s frozen; MLP [1,64,64,1] SiLU.
Poisson PINN (exp5)	30×30 interior grid, 200 boundary points, $\alpha = 0.01$; 40 LBFGS steps; test MSE on 4,000 random points; KAN G = 5, MLPs with tanh.

Threats to validity. Single seeds, short optimisation, one CPU core, and LBFGS for both model families mean that absolute numbers and small differences are not conclusive. The qualitative outcomes (symbolic recovery, parameter-efficient PDE solve, spline locality) are robust in the sense that they differ from their baselines by large factors.

Appendix D. Additional Tables from the Paper

D.1 Knot signature: symbolic formulas (paper Table 4, summarised)

ID	Formula / origin	Source	Test accuracy
A	$\lambda \cdot \mu_r / (\mu_r^2 + \mu_i^2)$ (the slope)	Human (DeepMind)	83.1%
B	Compact expression in sin, $\lvert\cdot\rvert$, Gaussian bumps	[3,1] KAN	62.6%
C	tan(·) of a sum of Gaussian bumps in μ_i, λ, μ_r	[3,1,1] KAN	71.9%
D	Nested exponentials and arctan terms	[3,2,1] KAN	84.0%
E	$4.76\ \lambda\ \mu_r / (3.09\ \mu_i + 6.05\ \mu_r^2 + 3.54\ \mu_i^2)$	[3,2,1] KAN + Padé	82.8%
F	Ratio using only μ_r and λ	Two [3,1] KANs	77.8%

Larger KANs give higher accuracy and more complex formulas (B → C → D), so KANs expose a whole Pareto frontier of accuracy versus simplicity. Adding an inductive bias (Padé form, or separate numerator and denominator networks) brings the discovered formula close to the human one.

D.2 Anderson localization accuracies (paper Table 5, summarised)

System	Method	Accuracy	Note
GAAM	Theory: $\alpha E + 2\lambda - 2 = 0$	99.2%	ground truth
GAAM	KAN auto (snap to linear/quadratic)	99.0%	small terms prunable
MAAM	Theory: $E + \exp(p) - \lambda \cosh p = 0$	98.6%	ground truth
MAAM	KAN auto (large library)	97.1%	complicated formula
MAAM	KAN manual, steps 2–3	97.7%	partly symbolic
MAAM	KAN manual, step 4A (cosh)	96.6%	simpler
MAAM	KAN manual, step 4B (quadratic)	95.4%	simplest

D.3 Hyperparameter effects on interpretability (paper Appendix C)

- Entropy penalty is needed to reduce the number of active edges; without it many duplicate functions remain.
- Results depend on random seed: an unlucky seed can leave a pruned network larger than necessary.
- Overall penalty λ effectively controls sparsity.
- Grid size G has a subtle effect: if G is too small each activation is not expressive enough and the network uses an 'ensembling' strategy that is harder to interpret.
- Spline order k has only a subtle, seed-like effect in the toy example.

D.4 Learnable activation networks (LANs) and image fitting

Appendix B of the paper introduces LANs: MLPs whose node activations are learnable splines. They have $N^2L + NLG$ parameters (little overhead over an MLP), are conceptually simpler and scale better in width than KANs, but are less interpretable (weight matrices remain) and less accurate. On synthetic examples the LAN diagrams are hard to read, which the authors take as evidence that removing linear weight matrices, by putting activations on edges, is necessary for interpretability. As a practical use, a SIREN network (sine activations) fitting an image can be converted to a LAN with a trainable spline added to the sine base function, improving PSNR with about 4% extra parameters ($G = 5$, $N = 128$).

D.5 Related work and discussion points

The paper situates KANs among earlier Kolmogorov-network papers (most use the original depth-2 width-(2n+1) form and predate back-propagation), generalised additive models, neural additive models, learnable-activation networks (polynomial, spline, SiLU-search, neural-network activations), symbolic regression tools (Eureqa, GPLearn, PySR, AI Feynman, EQL, OccamNet) and mechanistic interpretability, where KANs belong to the 'active' category of intrinsically interpretable designs. The discussion lists future directions: a generalised Kolmogorov–Arnold theorem defining 'KA depth', alternative local kernels (radial basis functions), adaptive grids, multi-head KANs for efficiency (activations shared within groups to exploit batching), hybrids with MLPs, and applications to Navier–Stokes, density functional theory and transformers.

References

[1] Z. Liu et al., "KAN: Kolmogorov–Arnold Networks," ICLR 2025, arXiv:2404.19756.

[2] A. N. Kolmogorov, "On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition," Dokl. Akad. Nauk, 114, 1957.

[3] K. Hornik, M. Stinchcombe, H. White, "Multilayer feedforward networks are universal approximators," Neural Networks, 2(5), 1989.

[4] U. Sharma and J. Kaplan, "A neural scaling law from the dimension of the data manifold," arXiv:2004.10802, 2020.

[5] E. J. Michaud, Z. Liu, M. Tegmark, "Precision machine learning," Entropy 25(1), 2023.

[6] T. Poggio, A. Banburski, Q. Liao, "Theoretical issues in deep networks," PNAS 117(48), 2020.

[7] A. Davies et al., "Advancing mathematics by guiding human intuition with AI," Nature 600, 2021.

[8] M. Raissi, P. Perdikaris, G. E. Karniadakis, "Physics-informed neural networks," J. Comput. Phys. 378, 2019.

[9] C. de Boor, A Practical Guide to Splines, Springer, 1978.